

CERTIFICACIÓN UNIVERSITARIA



UNC



FCEyN



Manual para el
alumno

Circle CI



WE WANT
SKILLS!

mundosE

Manual para el alumno: CircleCI.....	2
Objetivo del encuentro:.....	2
Contenido a desarrollar:.....	2
1. Introducción a CircleCI y al paradigma CI/CD.....	3
1.1 De la integración manual al despliegue continuo.....	3
1.2 Beneficios demostrables: datos de DORA.....	3
1.3 CircleCI en el ecosistema de herramientas.....	3
1.4 Caso de estudio: <i>fintech</i> rioplatense.....	4
1.5 Conceptos clave.....	4
2. Alta de cuenta y vinculación del repositorio.....	5
2.1 Creación de tu cuenta CircleCI.....	5
2.2 Configuración inicial del proyecto.....	5
2.3 Manejo de submódulos y grandes repositorios.....	6
2.4 Primer <i>pipeline</i> : análisis detallado.....	6
2.5 Tips de seguridad.....	7
3. Fundamentos del archivo config.yml.....	7
3.1 Desglose de la sintaxis YAML.....	7
3.2 Reutilización con <i>commands</i>	8
3.3 Parámetros y matrices.....	8
3.4 Estrategia de caché.....	9
3.5 Anti- <i>patterns</i> y cómo evitarlos.....	9
4. Pipelines avanzados y orbs.....	10
4.1 Introducción a <i>orbs</i> y su valor.....	10
4.2 Publicar tu propio <i>orb</i> paso a paso.....	10
4.3 Paralelismo y <i>test splitting</i>	10
4.4 <i>Fan-out / Fan-in</i>	10
4.5 Optimización de créditos y costos.....	11
5. Integraciones y despliegues continuos.....	11
5.1 Contenedores y <i>registries</i>	11
5.2 Estrategias <i>blue/green</i> y <i>canary</i>	12
5.3 Observabilidad y alertas.....	12
5.4 <i>Rollbacks</i> automáticos.....	12
Bibliografía.....	13

Este manual es un complemento de los encuentros sincrónicos de cada módulo. En este presentamos un resumen de cada tema tratado en el encuentro, así como también, recursos adicionales para poder profundizar sobre los conceptos vistos.

Manual para el alumno: CircleCI

Objetivo del encuentro:

Al finalizar este encuentro, estarás en capacidad de configurar, ejecutar y optimizar *pipelines* de integración y entrega continuas en CircleCI, comprendiendo la sintaxis del archivo `config.yml`, el uso de *orbs*, la gestión de secretos y las mejores prácticas para reducir el tiempo de construcción y de prueba.

Contenido a desarrollar:

- Introducción a CI/CD y posicionamiento de CircleCI.
- Arquitectura de CircleCI: SaaS vs. Server (Self-Hosted Runner).
- Vinculación de proyectos: GitHub, GitLab, Bitbucket.
- Estructura del archivo `config.yml`.
- Orbs: reutilización y publicación.
- Manejo de variables y secretos.
- Integraciones y despliegue continuo.

1. Introducción a CircleCI y al paradigma CI/CD

1.1 De la integración manual al despliegue continuo

La historia de la ingeniería de *software* está marcada por una tensión constante entre la velocidad de entrega y la calidad de los productos generados. A finales de la década de 1990, el aumento en la complejidad de los sistemas y la popularización de metodologías ágiles revelaron un cuello de botella recurrente: el *merge hell*, o “infierno de las integraciones”. Equipos que trabajaban en ramas aisladas durante semanas terminaban enfrentándose a conflictos masivos de código, *builds* rotos y retrasos en la salida a producción. Kent Beck y Martin Fowler, referentes de eXtreme Programming (XP), observaron que la solución no radicaba en integrar menos, sino en integrar mucho más seguido. **Así nació la Integración Continua (CI): un proceso automatizado en el que cada cambio se compila y testea en un entorno limpio, garantizando que el tronco principal del repositorio se mantenga siempre en estado “verde”.**

Pero integrar código sin entregarlo al usuario final resulta insuficiente. La lógica de negocio y los requisitos de mercado cambian a un ritmo vertiginoso; acortar el *lead time* entre la concepción de una funcionalidad y su uso real se volvió crucial. De allí surge la Entrega Continua (CD), que automatiza la empaquetación y deja artefactos listos para ser desplegados en producción con un solo “clic”. Cuando ese clic también se automatiza mediante *gates* de calidad y políticas de *feature flags*, hablamos de Continuous Deployment. Estas prácticas no son meros procesos técnicos: implican un cambio cultural en el que desarrollo, operaciones y negocio colaboran para entregar valor de forma iterativa y frecuente.

1.2 Beneficios demostrables: datos de DORA

Los reportes de la firma DORA (DevOps Research and Assessment), actualmente parte de Google Cloud, cuantifican la correlación entre CI/CD y rendimiento organizacional. Según el informe 2024, los equipos “Elite” despliegan **46 veces más frecuentemente** que los de bajo desempeño, mientras que su tiempo medio de recuperación ante fallos (MTTR) es un **2 600 % más rápido**. Además, presentan una tasa de cambios fallidos **7 veces menor**. Estos números demuestran que automatizar y acortar ciclos no compromete la calidad; por el contrario, la mejora.

1.3 CircleCI en el ecosistema de herramientas

Desde su aparición en 2011, CircleCI se posicionó como una plataforma SaaS-first que abstrae la infraestructura de CI/CD. A diferencia de Jenkins —donde la

instalación y el mantenimiento de *plugins* recaen en el equipo—, CircleCI ofrece contenedores efímeros, un *marketplace* de *orbs* (módulos reutilizables) y métricas incorporadas sin configuración adicional. Entre sus ventajas destacan:

- **Velocidad:** contenedores precargados con compiladores, SDK y *linters* reducen el tiempo de arranque.
- **Paralelismo:** la directiva `parallelism` divide *suites* de pruebas en múltiples nodos.
- **Elasticidad:** escalado horizontal automático sin intervención manual.
- **Marketplace:** más de 4 000 *orbs* certificados aceleran integraciones con AWS, GCP, Docker Hub, Slack, Snyk, etc.
- **Modelo de costos por créditos:** optimizable mediante caché y parámetros de recurso.

1.4 Caso de estudio: *fintech* rioplatense

Revisemos el caso “PagoRápido”, una *fintech* argentina que procesa pagos online. Antes de CircleCI, utilizaban Jenkins on-prem con un único *build master* que demandaba mantenimiento constante y generaba cuellos de botella. Las *builds* demoraban 25 min, los *tests end-to-end* se ejecutaban de noche y un fallo en producción tardaba horas en revertirse. Después de migrar a CircleCI, incorporaron:

1. **Pipelines paralelos** (pruebas unitarias, integración y e2e en nodos separados).
2. **Test Splitting** basado en *timings* históricos.
3. `**Orb **aws-ecr` para publicar imágenes Docker y `**orb **aws-ecs` para actualizar servicios.
4. **Contextos** con OIDC para obtener credenciales temporales en AWS.

Resultado: *builds* de 7 min, 3 despliegues diarios sin *downtime* y una reducción del 80 % en tickets de *rollback*. Lo más importante fue el impacto cultural: *marketing* pudo lanzar campañas alineadas con *releases* semanales y el equipo de soporte disminuyó tiempos de respuesta gracias a despliegues de *hotfix* en menos de una hora.

1.5 Conceptos clave

Término	Definición breve
Pipeline	Conjunto de <i>workflows</i> ejecutados tras un evento de VCS.
Workflow	Grafo de <i>jobs</i> con dependencias y paralelismo.
Job	Unidad atómica que corre en un <i>executor</i> (Docker, VM, macOS, Windows).

Step	Comando individual dentro de un <i>job</i> .
Orb	Paquete versionado de comandos, <i>jobs</i> y <i>executors</i> reutilizables.

Recursos para profundizar:

CircleCI – What is CI/CD?

- <https://circleci.com/ci-cd/>

State of DevOps Report 2024 (DORA):

1. <https://dora.dev/research/2024/dora-report/>
2. https://services.google.com/fh/files/misc/2024_final_dora_report.pdf

2. Alta de cuenta y vinculación del repositorio

2.1 Creación de tu cuenta CircleCI

Acceder a CircleCI es tan sencillo como autorizar tu usuario de GitHub o GitLab mediante OAuth 2.0. Al hacerlo, CircleCI solicita permisos de lectura (**contents:read**), escritura de *checks* (**checks:write**) y gestión de *pull requests* (**pull_requests:write**). Es fundamental revisar estos *scopes* para cumplir políticas de seguridad internas. Para proyectos corporativos se recomienda:

- Crear un usuario “bot” con acceso limitado a los repositorios esenciales.
- Habilitar 2FA tanto en GitHub/GitLab como en CircleCI.
- Restringir las *organizations* visibles en **Organization Settings → VCS**.

Una vez autenticado, el *dashboard* mostrará la lista de repos habilitados. Selecciona “**Set Up Project**” y deja que CircleCI genere un *pipeline* de prueba (“Hello World”) si tu repo aún no tiene **.circleci/config.yml**.

2.2 Configuración inicial del proyecto

CircleCI crea un *pipeline* por cada *push* o *pull request*. Para modificar esta conducta puedes usar filtros en **workflows** o activar “**Only build pull requests**” en

Project Settings. Es una buena práctica empezar con un *pipeline* mínimo:

```
version: 2.1
jobs:
  test:
    docker:
      - image: cimg/python:3.12
    steps:
      - checkout
      - run: pip install -r requirements.txt
      - run: pytest -q
workflows:
  ci:
    jobs: [ test ]
```

Sube este archivo y observa cómo CircleCI detecta el *commit* y arranca la *build*. Si tu repo es privado, CircleCI creará automáticamente una **Deploy Key** de solo lectura para clonar el código.

2.3 Manejo de submódulos y grandes repositorios

Para proyectos que utilizan submódulos Git, agrega:

```
- checkout:
  submodules: true
```

Si tus submódulos apuntan a repos privados de otra organización, entonces necesitas **User Key** o **SSH Key** adicional. Configúrala en **Project Settings → SSH Keys**.

Para repositorios de gran tamaño (>1 GB) conviene habilitar el *plugin* **git-lfs**. CircleCI soporta LFS de forma nativa cuando el repo ya lo utiliza. Recuerda limpiar artefactos viejos para no exceder el límite gratuito de 30 GB de *storage*.

2.4 Primer *pipeline*: análisis detallado

Al abrir el *pipeline* en la UI verás varias pestañas:

1. **Jobs:** lista cada *job* con su estado.
2. **Steps:** dentro de un *job*, muestra la salida completa de cada comando.

3. **Artifacts**: archivos subidos con `store_artifacts`.
4. **Tests**: reportes JUnit reconocidos automáticamente.
5. **Insights**: métricas históricas de duración y tasas de éxito.

Un círculo verde indica éxito; rojo, fallo; amarillo, ejecución; gris, cancelado. Con “**Rerun job with SSH**” puedes depurar accediendo al contenedor como si fuera tu máquina.

2.5 Tips de seguridad

- No imprimas *tokens* en consola. Si es imprescindible *loggear* contenido sensible, tápalo con `CircleCI CLI add_mask`.
- Guarda *secrets* en **Project** → **Environment variables** o en **Contexts**; nunca en el YAML.
- Revisa la sección **Audit Logs** (disponible en planes pagos) para detectar usos indebidos.

Recursos para profundizar:

Getting Started – CircleCI Docs: <https://circleci.com/docs/getting-started/>

Securing OAuth Apps: <https://docs.github.com/en/apps/oauth-apps> > <https://docs.github.com/en/apps/oauth-apps/building-oauth-apps/authorizing-oauth-apps>

3. Fundamentos del archivo `config.yml`

3.1 Desglose de la sintaxis YAML

CircleCI utiliza YAML versión 1.2. Cada bloque respeta sangrado de dos espacios. Un error de indentación anula la *build*; por eso la CLI `circleci config validate` es tu primera línea de defensa.

- `version: 2.1` habilita *orbs*.
- `orbs`: clave global. Formato `alias: namespace/orb@version`.
- `executors`: define contenedor o VM con *tag* de imagen.
- `commands`: macros paramétricos.
- `jobs`: unidades con `steps`.
- `workflows`: DAGs que orquestan *jobs*.

Ejemplo mínimo, pero real:


```

version: 2.1
jobs:
  lint:
    docker:
      - image: cimg/node:lts
    steps:
      - checkout
      - run: npm run lint
workflows:
  default: { jobs: [ lint ] }

```

3.2 Reutilización con *commands*

Para evitar duplicidad:

```

commands:
  restore_node_modules:
    steps:
      - restore_cache:
          keys:
            - v1-npm-{{ checksum "package-lock.json" }}
            - v1-npm-
      - run: npm ci
      - save_cache:
          key: v1-npm-{{ checksum "package-lock.json" }}
          paths: ~/.npm

```

Llámallo en cualquier *job* con - `restore_node_modules`.

3.3 Parámetros y matrices

Puedes declarar parámetros en *jobs* o *commands*:

```

commands:
  run_tests:
    parameters:
      db_url:
        type: string
    steps:
      - run: pytest --db << parameters.db_url >>

```

Las matrices (`matrix:`) te permiten combinar parámetros y generar *N jobs*. Es ideal para validar múltiples versiones de lenguaje y SO.

3.4 Estrategia de caché

CircleCI ofrece `save_cache` y `restore_cache` con claves jerárquicas. Una estrategia robusta:

```

- restore_cache:
  keys:
    - v1-pip-{{ checksum "requirements.txt" }}
    - v1-pip-

```

Si el *checksum* no coincide, busca la *prefix key* `v1-pip-` para usar la última caché válida. Recuerda incrementar el prefijo (`v2-...`) cuando cambies la estrategia.

3.5 Anti-patterns y cómo evitarlos

- **YAML duplicado** → usa *commands/orbs*.
- **Steps gigantes** en un único *job* → divide y conquista.
- **Variables hardcodedas** → pasa valores vía parámetros o *contexts*.
- **Faltas de comentarios** → documenta bloques complejos.

Recursos para profundizar:

Configuration Reference 2.1: <https://circleci.com/docs/configuration-reference/>

4. Pipelines avanzados y orbs

4.1 Introducción a orbs y su valor

Un **orb** es al YAML lo que un paquete *npm* es al Javascript: encapsula lógica reusable, versionada y firmada. En proyectos corporativos puedes tener un **namespace privado** con *orbs* internos que implementen políticas de seguridad, *linters* y plantillas de despliegue.

4.2 Publicar tu propio orb paso a paso

1. `circleci orb init my-orb` genera estructura.
2. Desarrolla tus *commands* y *jobs* en `/src`.
3. `circleci orb pack src > orb.yml` empaqueta.
4. `circleci orb validate orb.yml`.
5. `circleci orb publish orb.yml myorg/my-orb@dev:alpha`.
6. Pruebas en proyecto piloto → `circleci orb publish promote myorg/my-orb@dev:alpha 1.0.0`.

4.3 Paralelismo y test splitting

En CircleCI, `parallelism: N` crea N contenedores idénticos. Para dividir *tests*, usa:

```
pytest $(circleci tests split --split-by=timings)
```

La plataforma analiza los tiempos históricos y reparte archivos equitativamente.

4.4 Fan-out / Fan-in

Puedes *fan-out* a múltiples *jobs* y converger en uno que requiere resultados previos:

```
workflows:
  build-test-deploy:
    jobs:
      - build
      - test_unit:
          requires: [ build ]
      - test_e2e:
          requires: [ build ]
      - deploy:
          requires: [ test_unit, test_e2e ]
```

4.5 Optimización de créditos y costos

- Cache intensivamente.
- Cancela *pipelines* con la API cuando un nuevo *commit* invalide el anterior.
- Usa `resource_class: small` para *linters* y grandes solo donde aporte rendimiento.

Recursos para profundizar:

Orb Registry

- <https://circleci.com/developer/orbs/>
- <https://circleci.com/docs/orb-intro/>

5. Integraciones y despliegues continuos

5.1 Contenedores y *registries*

Para Docker Hub:

```

orbs:
  docker: circleci/docker@2
jobs:
  build_push:
    docker/build:
      image: "$DH_USER/app"
      tag: "$CIRCLE_SHA1"

```

El orb maneja `docker login` usando los secrets `DOCKERHUB_USERNAME` y `DOCKERHUB_PASSWORD`.

En AWS:

```

orbs:
  aws-ecr: circleci/aws-ecr@9
  aws-ecs: circleci/aws-ecs@3

```

5.2 Estrategias *blue/green* y *canary*

Con AWS ECS puedes definir dos servicios (*blue* y *green*) y balancear tráfico con Route 53. CircleCI actualiza la tarea *green* y, tras pasar *health checks*, rota el CNAME. Para *canary*, desplegar al 10 % del clúster y monitorear con CloudWatch antes de escalar.

5.3 Observabilidad y alertas

Insights ofrece *dashboards* de duración y tasa de errores. Exportará métricas a Datadog vía orb `datadog`. Configura alertas Slack con orb `slack@4`. También, puedes enviar *traces* OpenTelemetry con `otel/collector@1` para correlacionar *builds* y latencia de *endpoints*.

5.4 Rollbacks automáticos

CircleCI no despliega directo a producción; tu *pipeline* debe incluir pasos de *health check*. Si falla, ejecuta `circleci-agent step halt` o un job `rollback` que revierte a la imagen previa.

Bibliografía

Forsgren, N.; Humble, J.; Kim, G. (2021) Accelerate. IT Revolution.

Fowler, M. (2022) Refactoring CI Pipelines. O'Reilly.

Humble, J.; Kim, G. (2000) Continuous Delivery. Addison-Wesley.

Rahman, S. (2023) Learning CircleCI. Packt.